

layout: forward target:
https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
title: Hands parent: MediaPipe Legacy Solutions
nav_order: 4

MediaPipe Hands

{: .no_toc }

► Table of contents

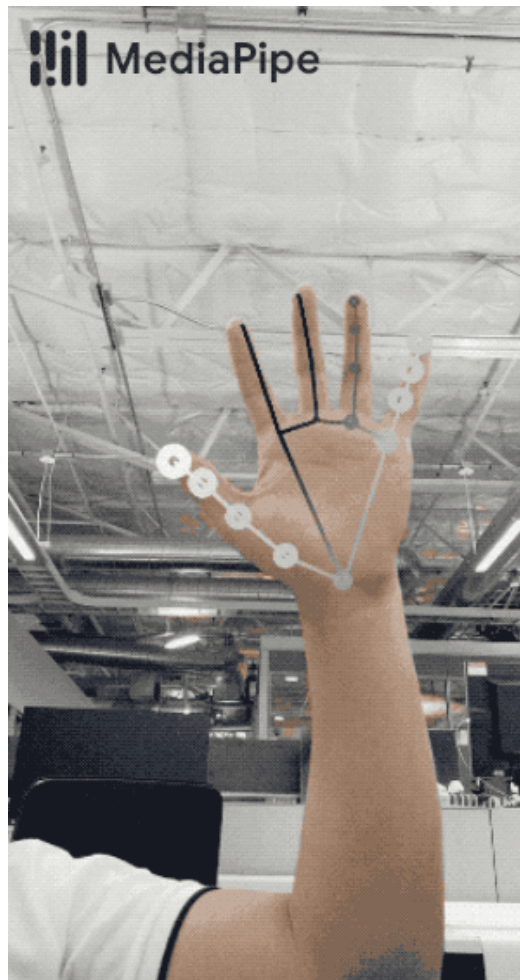
Attention: Thank you for your interest in MediaPipe Solutions. As of March 1, 2023, this solution was upgraded to a new MediaPipe Solution. For more information, see the [MediaPipe Solutions](#) site.

Overview

The ability to perceive the shape and motion of hands can be a vital component in improving the user experience across a variety of technological domains and platforms. For example, it can form the basis for sign language understanding and hand gesture control, and can also enable the overlay of digital content and information on top of the physical world in augmented reality. While coming naturally to people, robust real-time hand perception is a decidedly challenging computer vision task, as hands often occlude themselves or each other (e.g. finger/palm occlusions and hand shakes) and lack high contrast patterns.

MediaPipe Hands is a high-fidelity hand and finger tracking solution. It employs machine learning (ML) to infer 21 3D landmarks of a hand from just a single frame. Whereas current state-of-the-art approaches rely primarily on powerful desktop environments for inference, our method achieves

real-time performance on a mobile phone, and even scales to multiple hands. We hope that providing this hand perception functionality to the wider research and development community will result in an emergence of creative use cases, stimulating new applications and new research avenues.



hand_tracking_3d_android_gpu.gif | :

-----: | Fig 1. Tracked 3D hand landmarks are represented by dots in different shades, with the brighter ones denoting landmarks closer to the camera. |

ML Pipeline

MediaPipe Hands utilizes an ML pipeline consisting of multiple models working together: A palm detection model that operates on the full image and returns an oriented hand bounding box. A hand landmark model that operates on the cropped image region defined by the palm detector and

returns high-fidelity 3D hand keypoints. This strategy is similar to that employed in our [MediaPipe Face Mesh](#) solution, which uses a face detector together with a face landmark model.

Providing the accurately cropped hand image to the hand landmark model drastically reduces the need for data augmentation (e.g. rotations, translation and scale) and instead allows the network to dedicate most of its capacity towards coordinate prediction accuracy. In addition, in our pipeline the crops can also be generated based on the hand landmarks identified in the previous frame, and only when the landmark model could no longer identify hand presence is palm detection invoked to relocalize the hand.

The pipeline is implemented as a MediaPipe [graph](#) that uses a [hand landmark tracking subgraph](#) from the [hand landmark module](#), and renders using a dedicated [hand renderer subgraph](#). The [hand landmark tracking subgraph](#) internally uses a [hand landmark subgraph](#) from the same module and a [palm detection subgraph](#) from the [palm detection module](#).

Note: To visualize a graph, copy the graph and paste it into [MediaPipe Visualizer](#). For more information on how to visualize its associated subgraphs, please see [visualizer documentation](#).

Models

Palm Detection Model

To detect initial hand locations, we designed a [single-shot detector](#) model optimized for mobile real-time uses in a manner similar to the face detection model in [MediaPipe Face Mesh](#). Detecting hands is a decidedly complex task: our [lite model](#) and [full model](#) have to work across a variety of hand sizes with a large scale span (~20x) relative to the image frame and be able to detect occluded and self-occluded hands. Whereas faces have high contrast patterns, e.g., in the eye and mouth region, the lack of such features in hands makes it comparatively difficult to detect them reliably from their visual features alone. Instead, providing additional context, like arm, body, or person features, aids accurate hand localization.

Our method addresses the above challenges using different strategies. First, we train a palm detector instead of a hand detector, since estimating bounding boxes of rigid objects like palms and fists is significantly simpler than detecting hands with articulated fingers. In addition, as palms are smaller objects, the non-maximum suppression algorithm works well even for two-hand self-occlusion cases, like handshakes. Moreover, palms can be modelled using square bounding boxes (anchors in ML terminology) ignoring other aspect ratios, and therefore reducing the number of

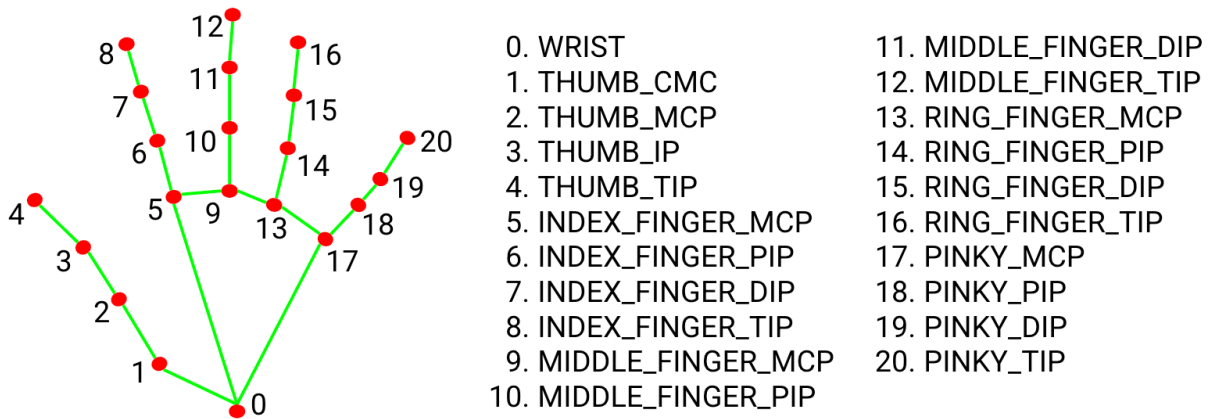
anchors by a factor of 3-5. Second, an encoder-decoder feature extractor is used for bigger scene context awareness even for small objects (similar to the RetinaNet approach). Lastly, we minimize the focal loss during training to support a large amount of anchors resulting from the high scale variance.

With the above techniques, we achieve an average precision of 95.7% in palm detection. Using a regular cross entropy loss and no decoder gives a baseline of just 86.22%.

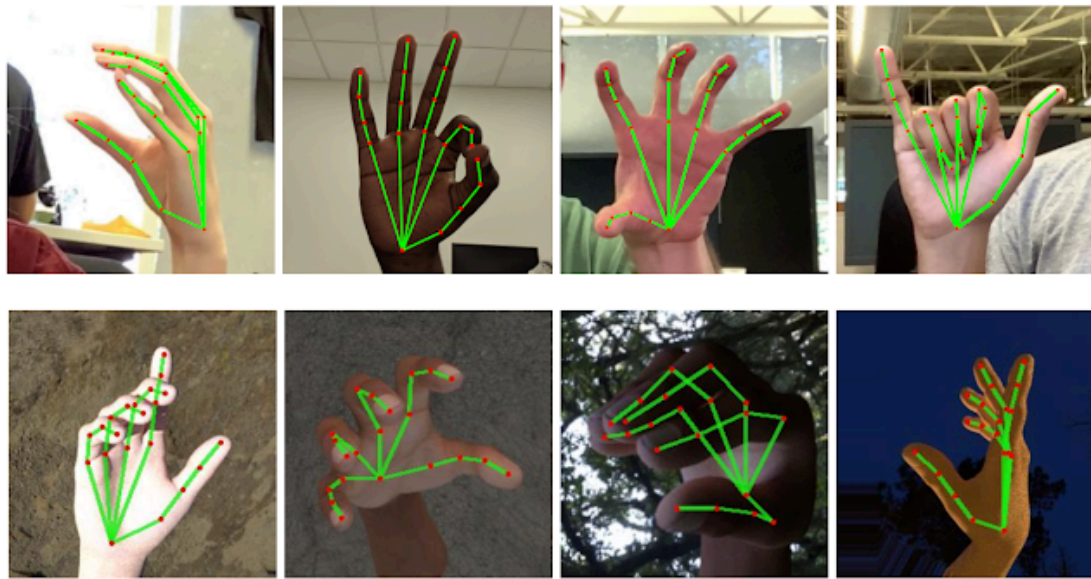
Hand Landmark Model

After the palm detection over the whole image our subsequent hand landmark [model](#) performs precise keypoint localization of 21 3D hand-knuckle coordinates inside the detected hand regions via regression, that is direct coordinate prediction. The model learns a consistent internal hand pose representation and is robust even to partially visible hands and self-occlusions.

To obtain ground truth data, we have manually annotated ~30K real-world images with 21 3D coordinates, as shown below (we take Z-value from image depth map, if it exists per corresponding coordinate). To better cover the possible hand poses and provide additional supervision on the nature of hand geometry, we also render a high-quality synthetic hand model over various backgrounds and map it to the corresponding 3D coordinates.



hand_landmarks.png | :-----: | Fig 2. 21 hand landmarks. |



hand_crops.png | :-----: | Fig 3. Top: Aligned hand crops passed to the tracking network with ground truth annotation. Bottom: Rendered synthetic hand images with ground truth annotation. |

Solution APIs

Configuration Options

Naming style and availability may differ slightly across platforms/languages.

static_image_mode

If set to `false`, the solution treats the input images as a video stream. It will try to detect hands in the first input images, and upon a successful detection further localizes the hand landmarks. In subsequent images, once all `max_num_hands` hands are detected and the corresponding hand landmarks are localized, it simply tracks those landmarks without invoking another detection until it loses track of any of the hands. This reduces latency and is ideal for processing video frames. If set to `true`, hand detection runs on every input image, ideal for processing a batch of static, possibly unrelated, images. Default to `false`.

max_num_hands

Maximum number of hands to detect. Default to `2`.

model_complexity

Complexity of the hand landmark model: `0` or `1`. Landmark accuracy as well as inference latency generally go up with the model complexity. Default to `1`.

min_detection_confidence

Minimum confidence value (`[0.0, 1.0]`) from the hand detection model for the detection to be considered successful. Default to `0.5`.

min_tracking_confidence:

Minimum confidence value (`[0.0, 1.0]`) from the landmark-tracking model for the hand landmarks to be considered tracked successfully, or otherwise hand detection will be invoked automatically on the next input image. Setting it to a higher value can increase robustness of the solution, at the expense of a higher latency. Ignored if `static_image_mode` is `true`, where hand detection simply runs on every image. Default to `0.5`.

Output

Naming style may differ slightly across platforms/languages.

multi_hand_landmarks

Collection of detected/tracked hands, where each hand is represented as a list of 21 hand landmarks and each landmark is composed of `x`, `y` and `z`. `x` and `y` are normalized to `[0.0, 1.0]` by the image width and height respectively. `z` represents the landmark depth with the depth at the wrist being the origin, and the smaller the value the closer the landmark is to the camera. The magnitude of `z` uses roughly the same scale as `x`.

multi_hand_world_landmarks

Collection of detected/tracked hands, where each hand is represented as a list of 21 hand landmarks in world coordinates. Each landmark is composed of `x`, `y` and `z`: real-world 3D coordinates in meters with the origin at the hand's approximate geometric center.

multi_handedness

Collection of handedness of the detected/tracked hands (i.e. is it a left or right hand). Each hand is composed of `label` and `score`. `label` is a string of value either `"Left"` or `"Right"`. `score` is the estimated probability of the predicted handedness and is always greater than or equal to `0.5` (and the opposite handedness has an estimated probability of `1 - score`).

Note that handedness is determined assuming the input image is mirrored, i.e., taken with a front-facing/selfie camera with images flipped horizontally. If it is not the case, please swap the handedness output in the application.

Python Solution API

Please first follow general [instructions](#) to install MediaPipe Python package, then learn more in the companion [Python Colab](#) and the usage example below.

Supported configuration options:

- [static_image_mode](#)
- [max_num_hands](#)
- [model_complexity](#)
- [min_detection_confidence](#)
- [min_tracking_confidence](#)

```

import cv2
import mediapipe as mp
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles
mp_hands = mp.solutions.hands

# For static images:
IMAGE_FILES = []
with mp_hands.Hands(
    static_image_mode=True,
    max_num_hands=2,
    min_detection_confidence=0.5) as hands:
    for idx, file in enumerate(IMAGE_FILES):
        # Read an image, flip it around y-axis for correct handedness output (see
        # above).
        image = cv2.flip(cv2.imread(file), 1)
        # Convert the BGR image to RGB before processing.
        results = hands.process(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))

        # Print handedness and draw hand landmarks on the image.
        print('Handedness:', results.multi_handedness)
        if not results.multi_hand_landmarks:
            continue
        image_height, image_width, _ = image.shape
        annotated_image = image.copy()
        for hand_landmarks in results.multi_hand_landmarks:
            print('hand_landmarks:', hand_landmarks)
            print(
                f'Index finger tip coordinates: (',
                f'{hand_landmarks.landmark[mp_hands.HandLandmark.INDEX_FINGER_TIP].x * image_width}, '
                f'{hand_landmarks.landmark[mp_hands.HandLandmark.INDEX_FINGER_TIP].y * image_height})'
            )
            mp_drawing.draw_landmarks(
                annotated_image,
                hand_landmarks,
                mp_hands.HAND_CONNECTIONS,
                mp_drawing_styles.get_default_hand_landmarks_style(),
                mp_drawing_styles.get_default_hand_connections_style())
        cv2.imwrite(
            '/tmp/annotated_image' + str(idx) + '.png', cv2.flip(annotated_image, 1))
        # Draw hand world Landmarks.
        if not results.multi_hand_world_landmarks:
            continue
        for hand_world_landmarks in results.multi_hand_world_landmarks:
            mp_drawing.plot_landmarks(
                hand_world_landmarks, mp_hands.HAND_CONNECTIONS, azimuth=5)

# For webcam input:
cap = cv2.VideoCapture(0)
with mp_hands.Hands(
    model_complexity=0,
    min_detection_confidence=0.5,

```



```

min_tracking_confidence=0.5) as hands:
while cap.isOpened():
    success, image = cap.read()
    if not success:
        print("Ignoring empty camera frame.")
        # If loading a video, use 'break' instead of 'continue'.
        continue

    # To improve performance, optionally mark the image as not writeable to
    # pass by reference.
    image.flags.writeable = False
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    results = hands.process(image)

    # Draw the hand annotations on the image.
    image.flags.writeable = True
    image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            mp_drawing.draw_landmarks(
                image,
                hand_landmarks,
                mp_hands.HAND_CONNECTIONS,
                mp_drawing_styles.get_default_hand_landmarks_style(),
                mp_drawing_styles.get_default_hand_connections_style())
    # Flip the image horizontally for a selfie-view display.
    cv2.imshow('MediaPipe Hands', cv2.flip(image, 1))
    if cv2.waitKey(5) & 0xFF == 27:
        break
cap.release()

```

JavaScript Solution API

Please first see general [introduction](#) on MediaPipe in JavaScript, then learn more in the companion [web demo](#) and a [fun application], and the following usage example.

Supported configuration options:

- [maxNumHands](#)
- [modelComplexity](#)
- [minDetectionConfidence](#)
- [minTrackingConfidence](#)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <script src="https://cdn.jsdelivr.net/npm/@mediapipe/camera_utils/camera_utils.js"
crossorigin="anonymous"></script>
  <script src="https://cdn.jsdelivr.net/npm/@mediapipe/control_utils/control_utils.js"
crossorigin="anonymous"></script>
  <script src="https://cdn.jsdelivr.net/npm/@mediapipe/drawing_utils/drawing_utils.js"
crossorigin="anonymous"></script>
  <script src="https://cdn.jsdelivr.net/npm/@mediapipe/hands/hands.js" crossorigin="anonymous">
</script>
</head>

<body>
  <div class="container">
    <video class="input_video"></video>
    <canvas class="output_canvas" width="1280px" height="720px"></canvas>
  </div>
</body>
</html>
```

```

<script type="module">
const videoElement = document.getElementsByClassName('input_video')[0];
const canvasElement = document.getElementsByClassName('output_canvas')[0];
const canvasCtx = canvasElement.getContext('2d');

function onResults(results) {
  canvasCtx.save();
  canvasCtx.clearRect(0, 0, canvasElement.width, canvasElement.height);
  canvasCtx.drawImage(
    results.image, 0, 0, canvasElement.width, canvasElement.height);
  if (results.multiHandLandmarks) {
    for (const landmarks of results.multiHandLandmarks) {
      drawConnectors(canvasCtx, landmarks, HAND_CONNECTIONS,
        {color: '#00FF00', lineWidth: 5});
      drawLandmarks(canvasCtx, landmarks, {color: '#FF0000', lineWidth: 2});
    }
  }
  canvasCtx.restore();
}

const hands = new Hands({locateFile: (file) => {
  return `https://cdn.jsdelivr.net/npm/@mediapipe/hands/${file}`;
}});
hands.setOptions({
  maxNumHands: 2,
  modelComplexity: 1,
  minDetectionConfidence: 0.5,
  minTrackingConfidence: 0.5
});
hands.onResults(onResults);

const camera = new Camera(videoElement, {
  onFrame: async () => {
    await hands.send({image: videoElement});
  },
  width: 1280,
  height: 720
});
camera.start();
</script>

```

Android Solution API

Please first follow general [instructions](#) to add MediaPipe Gradle dependencies and try the Android Solution API in the companion [example Android Studio project](#), and learn more in the usage example below.

Supported configuration options:

- [staticImageMode](#)
- [maxNumHands](#)
- runOnGpu: Run the pipeline and the model inference on GPU or CPU.

Camera Input

```

// For camera input and result rendering with OpenGL.
HandsOptions handsOptions =
    HandsOptions.builder()
        .setStaticImageMode(false)
        .setMaxNumHands(2)
        .setRunOnGpu(true).build();
Hands hands = new Hands(this, handsOptions);
hands.setErrorListener(
    (message, e) -> Log.e(TAG, "MediaPipe Hands error:" + message));

// Initializes a new CameraInput instance and connects it to MediaPipe Hands Solution.
CameraInput cameraInput = new CameraInput(this);
cameraInput.setNewFrameListener(
    textureFrame -> hands.send(textureFrame));

// Initializes a new GLSurfaceView with a ResultGLRenderer<HandsResult> instance
// that provides the interfaces to run user-defined OpenGL rendering code.
// See
mediapipe/examples/android/solutions/hands/src/main/java/com/google/mediapipe/examples/hands/HandsResult
// as an example.
SolutionGLSurfaceView<HandsResult> glSurfaceView =
    new SolutionGLSurfaceView<>(
        this, hands.getGLContext(), hands.getGLMajorVersion());
glSurfaceView.setSolutionResultRenderer(new HandsResultGLRenderer());
glSurfaceView.setRenderInputImage(true);

hands.setResultListener(
    handsResult -> {
        if (result.multiHandLandmarks().isEmpty()) {
            return;
        }
        NormalizedLandmark wristLandmark =
            handsResult.multiHandLandmarks().get(0).getLandmarkList().get(HandLandmark.WRIST);
        Log.i(
            TAG,
            String.format(
                "MediaPipe Hand wrist normalized coordinates (value range: [0, 1]): x=%f, y=%f",
                wristLandmark.getX(), wristLandmark.getY()));
        // Request GL rendering.
        glSurfaceView.setRenderData(handsResult);
        glSurfaceView.requestRender();
    });

// The runnable to start camera after the GLSurfaceView is attached.
glSurfaceView.post(
    () ->
        cameraInput.start(
            this,
            hands.getGLContext(),
            CameraInput.CameraFacing.FRONT,

```

```
glSurfaceView.getWidth(),  
glSurfaceView.getHeight());
```

Image Input

```

// For reading images from gallery and drawing the output in an ImageView.
HandsOptions handsOptions =
    HandsOptions.builder()
        .setStaticImageMode(true)
        .setMaxNumHands(2)
        .setRunOnGpu(true).build();
Hands hands = new Hands(this, handsOptions);

// Connects MediaPipe Hands Solution to the user-defined ImageView instance that
// allows users to have the custom drawing of the output Landmarks on it.
// See
mediapipe/examples/android/solutions/hands/src/main/java/com/google/mediapipe/examples/hands/HandsResult
// as an example.
HandsResultImageView imageView = new HandsResultImageView(this);
hands.setResultListener(
    handsResult -> {
        if (result.multiHandLandmarks().isEmpty()) {
            return;
        }
        int width = handsResult.inputBitmap().getWidth();
        int height = handsResult.inputBitmap().getHeight();
        NormalizedLandmark wristLandmark =
            handsResult.multiHandLandmarks().get(0).getLandmarkList().get(HandLandmark.WRIST);
        Log.i(
            TAG,
            String.format(
                "MediaPipe Hand wrist coordinates (pixel values): x=%f, y=%f",
                wristLandmark.getX() * width, wristLandmark.getY() * height));
        // Request canvas drawing.
        imageView.setHandsResult(handsResult);
        runOnUiThread(() -> imageView.update());
    });
hands.setEventListener(
    (message, e) -> Log.e(TAG, "MediaPipe Hands error:" + message));

// ActivityResultLauncher to get an image from the gallery as Bitmap.
ActivityResultLauncher<Intent> imageGetter =
    registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        result -> {
            Intent resultIntent = result.getData();
            if (resultIntent != null && result.getResultCode() == RESULT_OK) {
                Bitmap bitmap = null;
                try {
                    bitmap =
                        MediaStore.Images.Media.getBitmap(
                            this.getContentResolver(), resultIntent.getData());
                    // Please also rotate the Bitmap based on its orientation.
                } catch (IOException e) {
                    Log.e(TAG, "Bitmap reading error:" + e);
                }
                if (bitmap != null) {

```

```
        hands.send(bitmap);
    }
}
});
Intent pickImageIntent = new Intent(Intent.ACTION_PICK);
pickImageIntent.setDataAndType(MediaStore.Images.Media.INTERNAL_CONTENT_URI, "image/*");
imageGetter.launch(pickImageIntent);
```

Video Input


```

// For video input and result rendering with OpenGL.
HandsOptions handsOptions =
    HandsOptions.builder()
        .setStaticImageMode(false)
        .setMaxNumHands(2)
        .setRunOnGpu(true).build();
Hands hands = new Hands(this, handsOptions);
hands.setErrorListener(
    (message, e) -> Log.e(TAG, "MediaPipe Hands error:" + message));

// Initializes a new VideoInput instance and connects it to MediaPipe Hands Solution.
VideoInput videoInput = new VideoInput(this);
videoInput.setNewFrameListener(
    textureFrame -> hands.send(textureFrame));

// Initializes a new GLSurfaceView with a ResultGLRenderer<HandsResult> instance
// that provides the interfaces to run user-defined OpenGL rendering code.
// See
mediapipe/examples/android/solutions/hands/src/main/java/com/google/mediapipe/examples/hands/HandsResult
// as an example.
SolutionGLSurfaceView<HandsResult> glSurfaceView =
    new SolutionGLSurfaceView<>(
        this, hands.getGLContext(), hands.getGLMajorVersion());
glSurfaceView.setSolutionResultRenderer(new HandsResultGLRenderer());
glSurfaceView.setRenderInputImage(true);

hands.setResultListener(
    handsResult -> {
        if (result.multiHandLandmarks().isEmpty()) {
            return;
        }
        NormalizedLandmark wristLandmark =
            handsResult.multiHandLandmarks().get(0).getLandmarkList().get(HandLandmark.WRIST);
        Log.i(
            TAG,
            String.format(
                "MediaPipe Hand wrist normalized coordinates (value range: [0, 1]): x=%f, y=%f",
                wristLandmark.getX(), wristLandmark.getY()));
        // Request GL rendering.
        glSurfaceView.setRenderData(handsResult);
        glSurfaceView.requestRender();
    });

ActivityResultLauncher<Intent> videoGetter =
    registerActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        result -> {
            Intent resultIntent = result.getData();
            if (resultIntent != null) {
                if (result.getResultCode() == RESULT_OK) {
                    glSurfaceView.post(
                        () ->

```

```

        videoInput.start(
            this,
            resultIntent.getData(),
            hands.getGLContext(),
            glSurfaceView.getWidth(),
            glSurfaceView.getHeight());
    }
}
});
Intent pickVideoIntent = new Intent(Intent.ACTION_PICK);
pickVideoIntent.setDataAndType(MediaStore.Video.Media.INTERNAL_CONTENT_URI, "video/*");
videoGetter.launch(pickVideoIntent);

```

Example Apps

Please first see general instructions for [Android](#), [iOS](#) and [desktop](#) on how to build MediaPipe examples.

Note: To visualize a graph, copy the graph and paste it into [MediaPipe Visualizer](#). For more information on how to visualize its associated subgraphs, please see [visualizer documentation](#).

Mobile

Main Example

- Graph: `mediapipe/graphs/hand_tracking/hand_tracking_mobile.pbtxt`
- Android target: ([or download prebuilt ARM64 APK](#))
`mediapipe/examples/android/src/java/com/google/mediapipe/apps/handtrackinggpu:handtrackinggpu`
- iOS target: `mediapipe/examples/ios/handtrackinggpu:HandTrackingGpuApp`

Tip: Maximum number of hands to detect/process is set to 2 by default. To change it, for Android modify `NUM_HANDS` in [MainActivity.java](#), and for iOS modify `kNumHands` in [HandTrackingViewController.mm](#).

Palm/Hand Detection Only (no landmarks)

- Graph: `mediapipe/graphs/hand_tracking/hand_detection_mobile.pbtxt`

- Android target: (or [download prebuilt ARM64 APK](#))

`mediapipe/examples/android/src/java/com/google/mediapipe/apps/handdetectiongpu:handdetectiongpu`

- iOS target: `mediapipe/examples/ios/handdetectiongpu:HandDetectionGpuApp`

Desktop

- Running on CPU
 - Graph: `mediapipe/graphs/hand_tracking/hand_tracking_desktop_live.pbtxt`
 - Target: `mediapipe/examples/desktop/hand_tracking:hand_tracking_cpu`
- Running on GPU
 - Graph: `mediapipe/graphs/hand_tracking/hand_tracking_desktop_live_gpu.pbtxt`
 - Target: `mediapipe/examples/desktop/hand_tracking:hand_tracking_gpu`

Tip: Maximum number of hands to detect/process is set to 2 by default. To change it, in the graph file modify the option of `ConstantSidePacketCalculator`.

Resources

- Google AI Blog: [On-Device, Real-Time Hand Tracking with MediaPipe](#)
- TensorFlow Blog: [Face and hand tracking in the browser with MediaPipe and TensorFlow.js](#)
- Paper: [MediaPipe Hands: On-device Real-time Hand Tracking \(presentation\)](#)
- [Models and model cards](#)
- [Web demo](#)
- [Fun application](#)
- [Python Colab](#)